

Self-Evolving AI: A Unified Framework for Generation, Evaluation, Memory, and Improvement

A Survey of Self-Improvement Loops in Large Language Models,
Evolutionary Computation, and Autonomous Agents

Self Evolve Research Team

<https://github.com/copizzah/awesome-evolution>

May 2026

Abstract

AI self-evolution is not a single technique but a convergence of five fundamental loops: specification-to-execution, search, evaluator, reflection, and population. This survey systematically reviews the emerging landscape of self-improving AI systems, spanning 19+ open-source projects and 17+ foundational papers across five research communities—AutoML, Neural Architecture Search (NAS), LLM self-improvement, evolutionary computation, and autonomous agent frameworks. We propose a unified taxonomy that classifies existing work along the five evolution loops, revealing that progress in each community independently rediscovers mechanisms that others have formalized. We analyze self-improvement methods at both inference time (Self-Refine, Reflexion, Self-Debug) and training time (SPIN, STaR, Constitutional AI), trace the emergence of LLM-driven evolutionary optimization from OPRO to AlphaEvolve, and map the frontier of agent self-evolution from fixed architectures to open-ended self-modifying systems (Darwin Gödel Machine, Gödel Agent, Absolute Zero). We identify critical gaps in current agent frameworks and position Self-Evolve as the missing evolution layer above existing orchestration frameworks.

Keywords: Self-Evolution, LLM Self-Improvement, Evolutionary Computation, Agent Evolution, AutoML, Neural Architecture Search, Autonomous Agents

Contents

1	Introduction	4
1.1	From Static AI to Self-Evolving AI	4
1.2	What is AI Self-Evolution?	4
1.3	Relation to Existing Concepts	4
1.4	Contributions	5
1.5	Paper Structure	5
1.6	Related Surveys	5
2	Taxonomy: Five Evolution Loops	5
2.1	Specification-to-Execution Loop	6
2.2	Search Loop	6
2.3	Evaluator Loop	7
2.4	Reflection Loop	7
2.5	Population Loop	8
2.6	Cross-Loop Comparison and Interactions	8

3	Self-Improvement Methods	9
3.1	Inference-Time Self-Improvement	9
3.1.1	Self-Refine	9
3.1.2	Reflexion	9
3.1.3	Self-Debug	10
3.2	Training-Time Self-Improvement	10
3.2.1	SPIN: Self-Play Fine-Tuning	10
3.2.2	STaR: Self-Taught Reasoner	10
3.2.3	Constitutional AI	10
3.2.4	ReST: Reinforced Self-Training	10
3.3	Inference-Time vs. Training-Time Trade-offs	11
3.4	Comparative Analysis	11
4	Evolutionary Code and Algorithm Discovery	11
4.1	LLM as Optimizer	11
4.1.1	OPRO: Optimization by PROMPTing	11
4.2	LLM as Evolutionary Operator	12
4.2.1	AlphaEvolve	12
4.2.2	LLaMEA: Evolving the Optimizer	12
4.3	Program Evolution and Mathematical Discovery	12
4.3.1	FunSearch	12
4.4	Open-Source Implementations	13
4.5	Challenges and Limitations	13
4.6	Broader Impacts	13
5	Agent Self-Evolution Systems	14
5.1	Agent Architecture Self-Evolution	14
5.1.1	ADAS: Automated Design of Agentic Systems	14
5.1.2	Gödel Agent	14
5.1.3	Darwin Gödel Machine	15
5.2	Agent Workflow Self-Evolution	15
5.3	Agent Memory and Learning	15
5.3.1	Agent Symbolic Learning	15
5.3.2	Absolute Zero	16
5.4	Multi-Agent Collaborative Evolution	16
5.5	Toward a Hierarchy of Self-Evolution	16
6	Agent Frameworks and Infrastructure	16
6.1	Framework Overview and Comparison	16
6.2	Evolution Capability Gaps	17
6.3	Self-Evolve Positioning: Evolution Layer Above Frameworks	18
7	AutoML and NAS Meets LLM	18
7.1	AutoML + LLM	18
7.2	NAS + LLM	18
7.3	The Text-to-ML Paradigm	19
7.4	Architecture as Genome: The NAS Metaphor	19
7.5	Unified Perspective with Self-Evolution	19
8	Research Landscape and Community	19
8.1	Core Research Groups	19
8.2	Academic Lineage	20
8.3	Cross-Community Convergence	20
8.4	Open-Source Ecosystem	20

9	Open Problems and Future Directions	20
9.1	The Evaluation Bottleneck	20
9.2	Memory Stability and Drift	21
9.3	Safety and Alignment in Self-Modifying Systems	21
9.4	Composability of Evolution Loops	22
9.5	From Research to Production	22
9.6	The Self-Evolve Roadmap	23
10	Conclusion	23
A	Benchmark Data Summary	23
B	Core Papers Analyzed	23

1 Introduction

The history of artificial intelligence is characterized by alternating waves of static, hand-designed systems and adaptive, self-improving ones. Early expert systems required manual knowledge engineering; statistical machine learning introduced automated parameter fitting; deep learning automated feature engineering; and now, large language models (LLMs) are beginning to automate their own improvement. We stand at the threshold of a new paradigm: **self-evolving AI**—systems that can generate, evaluate, remember, and improve upon their own behavior without explicit human intervention at each step.

1.1 From Static AI to Self-Evolving AI

The trajectory from static to self-evolving AI can be understood through expanding automation scopes:

1. **Static systems** (pre-2012): Hand-crafted rules, features, and architectures.
2. **Learned representations** (2012–2017): Deep learning automated feature engineering.
3. **Architecture search** (2017–2020): NAS and AutoML automated model and pipeline design.
4. **Prompt-driven systems** (2020–2023): LLMs enabled natural language task specification.
5. **Self-evolving systems** (2023–present): Systems modify their own code, prompts, workflows, and architectures through iterative improvement cycles.

1.2 What is AI Self-Evolution?

We define **AI self-evolution** as the capacity of an AI system to autonomously improve through iterative cycles involving:

- **Generation:** Producing candidate solutions, architectures, prompts, code, or behaviors.
- **Evaluation:** Scoring candidates against objectives, benchmarks, or deployed metrics.
- **Memory:** Storing successful and failed attempts for future reference.
- **Improvement:** Using accumulated experience to produce better candidates.

This definition deliberately spans multiple communities. A prompt optimization system (OPRO) and an agent architecture search system (ADAS) both exhibit self-evolution, despite operating on different substrates and using different mechanisms.

1.3 Relation to Existing Concepts

AI self-evolution intersects with but is distinct from: **continual learning** (which updates on data streams, not system structure), **online learning** (parameter updates from streaming data), **self-supervised learning** (training signals from data itself), and **meta-learning** (learning how to learn, but typically within fixed architectures).

1.4 Contributions

1. A unified taxonomy of **five evolution loops** across five research communities (Section 2).
2. Systematic analysis of **19+ open-source projects** with detailed comparisons.
3. A **cross-domain perspective** unifying AutoML, NAS, LLM self-improvement, evolutionary computation, and agent frameworks.
4. Analysis of the **researcher ecosystem** and citation networks (Section 8).
5. Identification of **open problems** and a production roadmap (Section 9).

1.5 Paper Structure

Section 2 presents our five-loop taxonomy. Section 3 reviews self-improvement methods. Section 4 covers evolutionary code optimization. Section 5 surveys agent self-evolution. Section 6 reviews agent frameworks. Section 7 examines AutoML and NAS. Section 8 maps the community. Section 9 discusses open problems. Section 10 concludes.

1.6 Related Surveys

The survey on self-evolution of LLMs [?] organizes LLM self-improvement into four stages but does not encompass evolutionary computation, agent self-modification, and AutoML/NAS. RISE [?] examines recursive self-improvement theoretically. Our survey provides a unified taxonomy spanning all five communities, analyzing 19+ implementations with concrete gaps.

2 Taxonomy: Five Evolution Loops

We propose that self-evolving AI systems can be understood through five fundamental evolution loops. Each represents a distinct pattern independently discovered by different research communities.

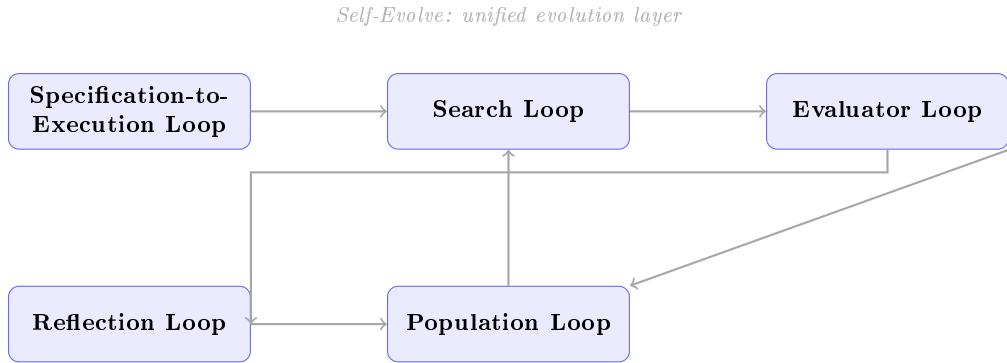


Figure 1: The five evolution loops and their interconnections.

2.1 Specification-to-Execution Loop

Translates natural language goals into runnable ML pipelines or agent workflows. **Representatives:** AutoML-GPT [?] uses GPT as a natural language bridge, converting task/model/data cards into complete ML pipelines. AutoML-Agent [?] decomposes AutoML into specialized agents for data retrieval, preprocessing, modeling, evaluation, and deployment with retrieval-augmented planning and multi-stage validation.

Loop dynamics: User specifies task → system plans pipeline → executes → evaluates → refines specification understanding. The main gap is **persistent memory**—most systems start from scratch for each new task.

2.2 Search Loop

Explores the space of possible solutions using LLMs as intelligent search operators. **Representatives:** OPRO [?] uses LLMs as general-purpose optimizers where solution-score history guides generation. Evo-Prompting [?] treats LLMs as evolutionary operators for architecture code. ADAS [?] automates search over agent design spaces. OpenEvolve [?] provides open-source evolutionary code optimization.

LLM-driven search differs from traditional optimization through the model’s ability to understand search space structure and propose semantically meaningful variations. **Key mechanisms:** temperature-controlled exploration, few-shot guidance from historical solutions, hash-based deduplication.

2.3 Evaluator Loop

Provides the fitness signal driving all other loops. Without reliable evaluation, generation is blind and search is random. **Representatives:** FunSearch [?] achieves the first improvement over Strassen’s algorithm in 56 years through LLM + automated evaluator. Self-Debug [?] uses execution feedback as self-repair signal.

Evaluation modalities: automated testing, benchmark suites (HumanEval, GSM8K, SWE-bench), execution traces, and deployment metrics. The evaluator loop is the **bottleneck** of self-evolution: as systems saturate benchmarks, marginal information gain decreases.

2.4 Reflection Loop

Converts failures into memory, feedback, and revised strategies. Unlike evaluators (scores), reflection produces *explanations* and *suggestions*. **Representatives:** Reflexion [?] introduces verbal reinforcement learning (91% HumanEval pass@1 vs GPT-4’s 80%). Self-Refine [?] demonstrates same-LLM generate-critique-rewrite. STaR [?] extends to training time with self-generated reasoning traces.

2.5 Population Loop

Maintains multiple candidates simultaneously, enabling selection, mutation, recombination, and specialization. **Representatives:** AlphaEvolve [?] uses MAP-Elites quality-diversity search with Gemini Flash + Pro, achieving 48 multiplications for 4×4 complex matrix multiplication (first Strassen improvement in 56 years), recovering 0.7% worldwide Google compute. LLaMEA [?] evolves optimizers themselves. Darwin Gödel Machine [?] maintains open-ended archives (SWE-bench: 20%→50%).

2.6 Cross-Loop Comparison

Table 1: Comparison of the five evolution loops.

Loop	What Evolves	Generator	Evaluator	Memory
Spec-to-Exec	ML pipelines	Planner + agents	Dataset metrics	Pipeline history
Search	Prompts, code, arch.	LLM as operator	Task scores	Candidate archive
Evaluator	Eval strategies	Test generators	Meta-evaluation	Failure patterns
Reflection	Reasoning	Self-critique LLM	Verbal feedback	Episodic memory
Population	Agent impls	LLM mutation	Fitness functions	Population archive

The most effective systems combine multiple loops. AlphaEvolve combines search + evaluator + population. Reflexion combines search + evaluator + reflection. Interactions between loops can be synergistic (strong evaluator improves reflection quality) or conflicting (population diversity may slow search convergence).

3 Self-Improvement Methods

3.1 Inference-Time Self-Improvement

3.1.1 Self-Refine

Self-Refine [?] demonstrates that a single LLM can generate, critique, and iteratively rewrite outputs at inference time. Applied to dialogue, code review, math reasoning, and sentiment reversal. The LLM’s evaluation ability often exceeds its generation ability—separating these enables iterative gap-closing. 2–3 refinement iterations typically yield diminishing returns.

3.1.2 Reflexion

Reflexion [?] introduces persistent linguistic feedback. Three models: Actor M_a , Evaluator M_e , Self-Reflection M_r . Core loop: $a_e = M_a(\text{task}, M_{1:e-1})$, $r_e = M_e(a_e)$, $f_e = M_r(a_e, r_e)$, $M_e = M_{e-1} \cup \{f_e\}$. Achieves 91% HumanEval pass@1 vs GPT-4’s 80%. The memory buffer acts as a “verbal gradient” replacing parameter updates.

3.1.3 Self-Debug

Self-Debug [?] uses execution feedback as self-repair signal. Unlike Self-Refine (internal critique) or Reflexion (linguistic feedback), Self-Debug grounds improvement in concrete runtime behavior—error messages, test outputs, and debugging information.

3.2 Training-Time Self-Improvement

3.2.1 SPIN

SPIN [?] improves SFT models through self-play: the model generates responses, a previous-iteration critic distinguishes model from human outputs, and the model fine-tunes to become indistinguishable from human.

3.2.2 STaR

STaR [?] generates chain-of-thought rationales, filters correct ones, and fine-tunes on filtered sets. “Reasoning generates reasoning”—successful traces become training data. Can leverage incorrect rationales by conditioning on correct answers for revision.

3.2.3 Constitutional AI

Constitutional AI [?] introduces AI feedback self-alignment. Models generate, critique against constitutional principles, and revise. Demonstrates that LLMs can serve as their own reward signal.

3.2.4 ReST

ReST [?] combines self-generation with RL: generate multiple solutions, filter by correctness, fine-tune on filtered set using expectation-maximization.

3.3 Inference vs. Training Trade-offs

- **Cost:** Inference-time cheaper per use but repeated; training-time expensive upfront but amortized.
- **Persistence:** Inference improvements ephemeral; training improvements persist in weights.
- **Generality:** Inference methods task-specific; training improvements can generalize.
- **Safety:** Inference processes visible in prompts; training modifications opaque.

3.4 Comparative Analysis

4 Evolutionary Code and Algorithm Discovery

4.1 LLM as Optimizer: OPRO

OPRO [?] pioneered LLMs as general-purpose optimizers. The **two-layer prompt architecture**: outer meta-prompt (optimizer LLM, temp=1.0) contains sorted instruction-score pairs + few-shot examples; inner task prompt (scorer LLM, temp=0.0) applies generated instructions. Key details: 200 steps, 8 candidates/step, MD5 deduplication, 100-bin score bucketization, wrong-question tracking for dynamic few-shot selection. Cross-task generalization: GSM8K (+8%), BBH, MMLU, linear regression, TSP.

Table 2: Comparison of self-improvement methods.

Method	Loop(s)	Train?	Feedback	Key Result
Self-Refine	Reflection	No	Self-critique	Iterative improvement
Reflexion	Reflect+Eval	No	Verbal memory	91% HumanEval
Self-Debug	Eval+Reflect	No	Execution traces	Runtime self-repair
SPIN	Population	Yes	Self-play	Weak→strong
STaR	Reflect+Eval	Yes	Self-rationales	Reasoning→reasoning
Const. AI	Reflection	Yes	Principles	Self-alignment
ReST	Search+Eval	Yes	Correctness filter	EM self-training

4.2 LLM as Evolutionary Operator

4.2.1 AlphaEvolve

AlphaEvolve [?] combines: (1) Dual-model architecture: Gemini Flash (breadth) + Pro (depth). (2) MAP-Elites quality-diversity: $\text{Archive}[b(p)] = p$ if $f(p) > f(\text{Archive}[b(p)])$. (3) Automated evaluators. (4) Evolution context adaptation.

Results: 48 multiplications for 4×4 complex matrix mult (first Strassen improvement in 56 years); re-discovered SOTA for 75% of 50+ math problems; recovered 0.7% worldwide Google compute; 23% FlashAttention speedup; 32% attention speedup.

4.2.2 LLaMEA

LLaMEA [?] evolves meta-heuristic algorithms—“improving the improver.” The LLM proposes optimization algorithms as code, evaluated on benchmarks. Best algorithms retained as parents.

4.3 FunSearch

FunSearch [?] (Nature) combines LLMs with automated evaluators for mathematical program evolution. Key insight: **evaluator-constrained program evolution surpasses one-shot generation**. Results: new cap set records, bin packing improvements.

4.4 Open-Source Implementations

OpenEvolve [?] (6,358 stars): community AlphaEvolve. SE-Agent: multi-trajectory revision/recombination. Science-CodeEvolve: scientific code evolution. SelfEvolve [?]: self-generated knowledge + self-debugging (DS-1000: 49.4→57.2, HumanEval: 74.39→85.98).

4.5 Challenges and Broader Impacts

Challenges include evaluation cost scaling, search space design, reward hacking, reproducibility of LLM-generated results, and scalability to large software systems. FunSearch’s mathematical discoveries and AlphaEvolve’s infrastructure optimizations demonstrate significant broader impacts, but raise concerns about research displacement and capability concentration.

5 Agent Self-Evolution Systems

5.1 Architecture Self-Evolution

5.1.1 ADAS

ADAS [?] automates search over agent architecture design spaces. The meta-agent proposes architectures, evaluates on tasks, and iteratively improves. Search space includes LLM call structure, prompt templates, tool integration, and control flow.

Table 3: Evolutionary code optimization systems.

System	Year	LLM Role	Strategy	Key Result
OPRO	2023	Optimizer	History-guided	+8% GSM8K
FunSearch	2024	Mutation	Program evolution	Nature publication
AlphaEvolve	2025	Mut+crossover	MAP-Elites QD	48-mult Strassen
LLaMEA	2024	Algorithm gen	EA on algorithms	Novel meta-heuristics
OpenEvolve	2024	Mutation	Evolutionary	Community impl.
SelfEvolve	2023	Gen+debug	Self-debug loop	+15.8% DS-1000

5.1.2 Gödel Agent

Gödel Agent [?]: self-referential framework using LLM-driven monkey patching. Loop: (1) Read own code, (2) LLM proposes patches, (3) Apply at runtime, (4) Validate, (5) Rollback if degraded. Self-referential: $A' = \text{LLM}(A.\text{code}, \text{feedback}, \text{objective})$. Safety: rollback, validation, objective constraints.

5.1.3 Darwin Gödel Machine

DGM [?]: combines Darwinian evolution with Gödel self-reference. Open-ended archive $A = \{a_1, \dots, a_n\}$. Sampling: $P(a_i) \propto f(a_i) \times \text{diversity_bonus}$. Modification: $a_{\text{child}} = \text{LLM}(a_{\text{parent}}.\text{code}, \text{edit})$. Selection: keep if improved. Results: SWE-bench 20%→50%, Polyglot 14.2%→30.7%. Open-endedness prevents convergence to single optimum.

5.2 Workflow Self-Evolution

AgentEvolver: evolutionary tool usage optimization. SE-Agent: multi-trajectory Revision/Recombination/Refinement.

5.3 Memory and Learning

5.3.1 Agent Symbolic Learning

Agent Symbolic Learning [?] (NeurIPS 2024): treats agents as symbolic networks. Algorithm: (1) Forward pass → trajectory τ , (2) Language loss $\mathcal{L}_{\text{lang}} = \text{LLM}(\mathcal{P}_{\text{loss}}(\tau))$, (3) Language gradient backpropagation layer-by-layer, (4) Weight update via PromptOptimizer, ToolOptimizer, PipelineOptimizer. Results: HotPotQA F1 39.1, MATH 60.7%, HumanEval 73.2%.

5.3.2 Absolute Zero

Absolute Zero [?] (NeurIPS 2025 Award): model simultaneously proposes and solves tasks using only self-generated data. Dual-role: Task Proposer → Task Solver → Code Executor → RL update. Eliminates human-curated data dependency entirely.

5.4 Multi-Agent Collaborative Evolution

AutoML-Agent [?]: multi-agent division of labor for full-pipeline AutoML. SelfEvolve [?]: self-generated knowledge + self-debugging pipeline.

5.5 Hierarchy of Self-Evolution

(1) Output optimization (OPRO, Self-Refine), (2) Workflow optimization (AgentEvolver), (3) Architecture optimization (ADAS, Gödel Agent), (4) Open-ended evolution (DGM, Absolute Zero). Each level subsumes the previous.

6 Agent Frameworks and Infrastructure

6.1 Framework Comparison

- **AutoGPT** [?]: Goal-driven autonomous agents, evaluation infrastructure.
- **MetaGPT** [?]: SOP-driven multi-agent collaboration, role specialization.
- **AutoGen** [?]: Conversational agents, human-in-the-loop, multi-agent workflows.
- **DSPy** [?]: Declarative LLM programming, prompt optimization as compilation.
- **CAMEL** [?]: Communicative role-playing agents.

Table 4: Agent framework comparison.

Framework	Orchestration	Memory	Tools	Evaluation	Evolution
AutoGPT	Goal-driven	Short-term	Yes	Basic	Limited
MetaGPT	SOP-driven	Process	Yes	Role-based	None
AutoGen	Conversation	Configurable	Yes	External	None
DSPy	Declarative	Pipeline	Yes	Built-in	Prompt opt
CAMEL	Role-play	Conversation	Limited	External	None

6.2 Evolution Capability Gaps

Current frameworks lack: (1) **Persistent evaluation**: no accumulated evaluation history across runs. (2) **Lineage and regression protection**: no tracking of which changes led to which outcomes. (3) **Cross-run memory**: agents start from scratch each run. (4) **Cost awareness**: no tracking of self-improvement ROI.

6.3 Self-Evolve: Evolution Layer Above Frameworks

Self-Evolve sits above existing frameworks as an evolution infrastructure, providing: evaluation service (persistent multi-dimensional scoring), lineage tracker (version control for self-modifying agents), evolution engine (five loops applied to prompts/code/tools/architecture), and memory service (cross-task persistent knowledge base).

7 AutoML and NAS Meets LLM

7.1 AutoML + LLM

AutoML-GPT [?]: natural language bridge for complete ML pipelines. AutoML-Agent [?]: multi-agent full-pipeline AutoML. Most production-ready self-evolution direction but lacks continuous evolution across runs.

7.2 NAS + LLM

EvoPrompting [?]: LLMs as evolutionary operators for architecture code. LLMatic [?]: LLM-guided quality-diversity architecture search. NADER [?]: multi-agent neural architecture design. Key metaphor: **architecture is the evolvable genome**.

7.3 Unified Perspective

AutoML evolves pipelines (search + specification loops); NAS evolves architectures (search + evaluator loops). Both benefit from reflection and population loops. Cross-community techniques transfer directly: MAP-Elites from robotics to agent architecture, reflexion from agents to AutoML, constitutional principles from safety to evolutionary systems.

8 Research Landscape and Community

8.1 Core Research Groups

Google/DeepMind: OPRO [?], FunSearch [?], AlphaEvolve [?]. **Stanford:** STaR [?] (Goodman lineage). **Princeton NLP:** Reflexion [?] (Narasimhan→Yao). **UBC:** DGM [?], ADAS [?] (Clune group). **Tsinghua:** Absolute Zero [?] (NeurIPS 2025 Award). **Anthropic:** Constitutional AI [?].

8.2 Academic Lineage

Goodman → Zelikman (self-taught reasoning); Narasimhan → Yao (agent reasoning); Clune → Hu, Lu, Zhang (open-ended evolution); W.Y. Wang → Yin (self-referential agents).

8.3 Cross-Community Convergence

AutoML’s “task card → pipeline” mirrors agent “goal → plan → execute”; NAS’s “mutation → fitness” mirrors evolutionary “LLM-as-operator”; self-improvement’s “generate → critique → refine” mirrors “act → reflect → retry.” The five loops are fundamental design-space attractors.

8.4 Open-Source Ecosystem

Table 5: Open-source projects in AI self-evolution.

Project	Stars	Evolution Mode	License
OpenEvolve	6,358	Search + Evaluator	Apache 2.0
aiwaves/Agents	5,928	Search + Eval + Agent	Apache 2.0
Reflexion	3,158	Search + Reflection	MIT
AgentEvolver	1,441	Search + Eval + Agent	Apache 2.0
Self-Refine	805	Reflection	MIT
SE-Agent	274	Search + Eval + Agent	MIT
CodeEvolve	97	Search + Evaluator	MIT
SCOPE	77	Search Loop	MIT
LLM-Self-Judge	43	Search + Eval + Agent	Apache 2.0
DARWIN	41	Search + Reflection	MIT

9 Open Problems and Future Directions

9.1 Evaluation Bottleneck

Benchmark saturation risks overfitting. As systems saturate HumanEval/GSM8K/SWE-bench, marginal information gain decreases. **Direction:** multi-dimensional evaluation, adversarial probing, meta-evaluation.

9.2 Memory Stability and Drift

Catastrophic forgetting, memory drift, unbounded growth. **Direction:** forgetting curves, experience replay, selective retention.

9.3 Safety and Alignment

Objective drift, reward hacking, unbounded modification, deceptive alignment. **Direction:** constitutional invariants, sandboxed execution, formal verification, interpretability tracking.

9.4 Loop Composability

Loop interactions poorly understood. **Direction:** formal composition frameworks, meta-evolution of loop configurations.

9.5 Production Deployment

Latency constraints, API costs, observability, version control, regulatory compliance. **Direction:** cost-aware infrastructure, regression gates, evolutionary version tracking.

9.6 Self-Evolve Roadmap

1. **Phase 1** (current): Instrumentation—evaluation layers, lineage tracking, cross-run memory.
2. **Phase 2:** Single-loop evolution with regression gates.
3. **Phase 3:** Multi-loop composition with resource allocation.
4. **Phase 4:** Open-ended evolution discovering new strategies.

10 Conclusion

This survey presented a unified view of AI self-evolution through five fundamental loops. Our analysis of 19+ projects and 17+ papers across five communities reveals striking convergence: different communities independently rediscover the same self-improvement mechanisms. Self-evolving AI is an *emergent capability* arising from the four primitives of generation, evaluation, memory, and improvement.

The frontier moves from fixed-architecture improvement toward open-ended self-modification (Darwin Gödel Machine, Gödel Agent, Absolute Zero). Significant challenges remain: evaluation bottleneck, memory stability, safety alignment, and loop composability. Self-Evolve is positioned as the missing evolution layer above existing frameworks, providing persistent evaluation, memory, lineage, and improvement. The convergence across communities suggests self-evolving AI is an active frontier with concrete results; the challenge is moving from isolated demonstrations to integrated, safe, reliable systems.

A Benchmark Data

B Core Papers

Table 6: Key benchmark results.

System	Benchmark	Metric	Score
Reflexion	HumanEval	pass@1	91%
GPT-4 baseline	HumanEval	pass@1	80%
SelfEvolve	HumanEval	pass@1	85.98%
SelfEvolve	DS-1000	pass@1	57.2
DGM	SWE-bench	Verified	20%→50%
DGM	Polyglot	Score	14.2%→30.7%
Agent Sym. Learn.	HotPotQA	F1	39.1
Agent Sym. Learn.	MATH	Accuracy	60.7%
Agent Sym. Learn.	HumanEval	pass@1	73.2%
AlphaEvolve	4×4 matrix	Mults	48
AlphaEvolve	FlashAttention	Speedup	23%
AlphaEvolve	Google compute	Recovery	0.7%
OPRO	GSM8K	Accuracy	+8%
FunSearch	Cap set	Result	New record

Table 7: Core papers analyzed.

#	Paper	Contribution	Relevance
1	Agent Symbolic Learning	Symbolic backprop	High
2	Darwin Gödel Machine	Open-ended evolution	Very High
3	Gödel Agent	Self-referential mod	High
4	ADAS	Agent design search	High
5	Reflexion	Verbal RL	Very High
6	Self-Refine	Self-feedback	High
7	Absolute Zero	Zero-data self-play	Very High
8	AlphaEvolve	Gemini + MAP-Elites	Very High
9	RISE	Recursive improvement	High
10	RAGen	RAG generation	Med-High
11	SelfEvolve	Code evolution	Very High
12	ReVeal	Verification-guided	High